

**Capstone Project Enhancement Plan**

Jason Barry

Southern New Hampshire University

CS-499-18502-M01 Computer Science Capstone

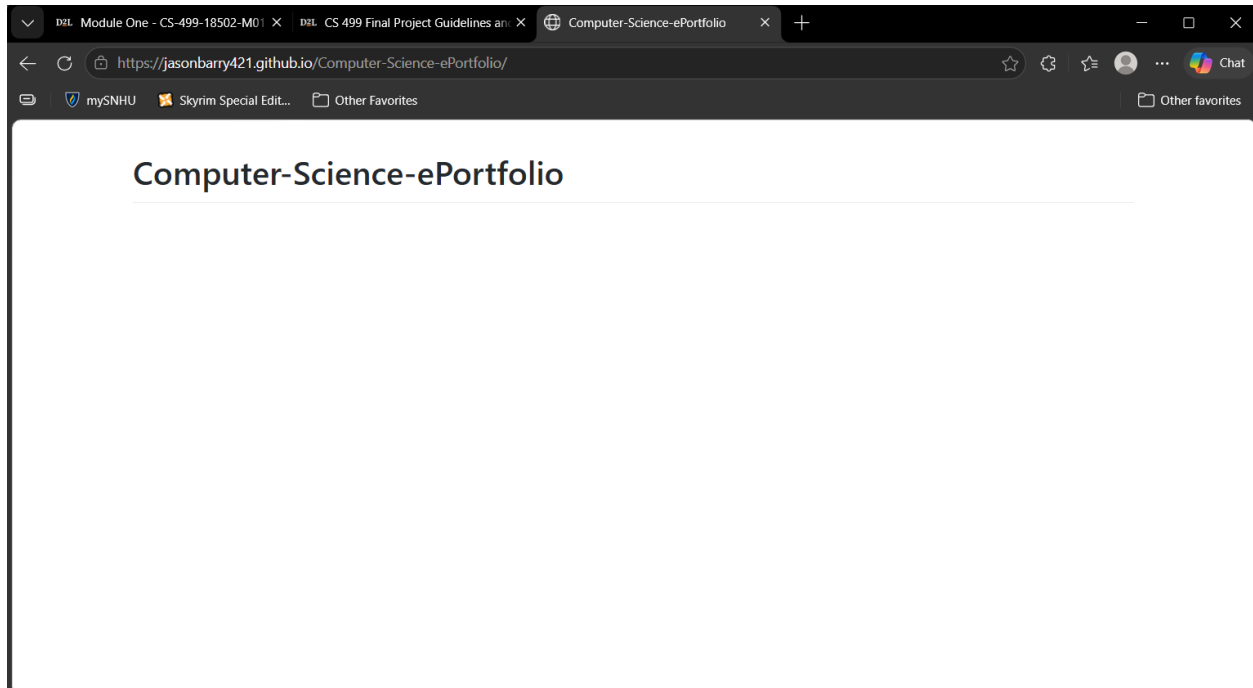
Professor Maciosek

7 May 2026

## **I. Self-Introduction**

I, Jason Barry, have been enrolled in the Computer Science program for approximately four years. During this time, I have developed key concepts and skills including designing mobile applications with structured user interfaces, working with databases and data modeling concepts, and applying industry-standard software design principles. Through my capstone project, I intend to demonstrate how these skills relate to software design and architecture by improving the structure and usability of a mobile application, implementing algorithmic solutions for time-based event organization and conflict handling, and redesigning the application's data layer to support secure, cloud-based multi-user storage. These improvements serve as an applied example of system design, data management, and computational problem-solving. Although I do not yet have a fully defined specialization within Computer Science, my experience in the program has helped me develop an interest in software development and application design, particularly in building user-focused systems with structured data and scalable architecture. This experience has also provided practical exposure to the types of design and development decisions commonly encountered in software development roles and has helped prepare me for the responsibilities of an entry-level software developer.

## **II. ePortfolio Set Up:**



### III. Enhancement Plan:

|                                                                           |
|---------------------------------------------------------------------------|
| Project used during Enhancements                                          |
| <a href="#">CS-360: Mobile Architecture and Engineering Final Project</a> |

#### Category One: Software Engineering and Design

For an improvement related to Software Design and Engineering, I will focus on my Final Project for CS-360: Mobile Architecture and Programming. This project involved the development of a mobile event calendar application that allows users to create and update events.

While the application's underlying database is capable of storing multiple events per day, the current user interface is limited to displaying only a single event per day.

In the existing implementation, when an event occurs on a given date, the corresponding day's background color changes from light gray to cyan. When the user selects that day, the event's details are displayed at the bottom of the screen, where they can be updated. However, this design fails to account for multiple events occurring on the same day. As a result, additional events are not displayed, preventing users from viewing or modifying them. This represents a limitation in the system's design rather than its data storage capabilities.

To address this issue, I will redesign the user interface to support a more scalable and modular display model. The updated design will introduce two coordinated views: a refined Monthly View for high-level overview and a new Daily View for detailed interaction.

In the Monthly View, each day cell will display up to two or three events in chronological order. If the number of events exceeds this limit, an overflow indicator (e.g., "+X more") will be displayed to inform the user that additional events exist.

| Monthly View Pseudocode                                                                                                                                                                                                                           |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>events = getEventsForDay(day) sort events by startTime  if events is not empty:     display up to MAX_VISIBLE_EVENTS (e.g., 2 or 3)      if total events &gt; MAX_VISIBLE_EVENTS:         display "+(total - MAX_VISIBLE_EVENTS) more"</pre> |

When a user selects a day, either by clicking the overflow indicator or the day itself, the application will transition to a Daily View. This view will present a vertical 24-hour timeline divided into hourly increments. Events will be displayed according to their start and end times, allowing overlapping events to be visualized simultaneously. Selecting an event within this view will navigate the user to the existing “Update Event” screen for modification.

This redesign improves the system’s overall software design by aligning the user interface with the underlying data model and introducing a layered interaction approach. The Monthly View provides a clean and scalable overview, while the Daily View enables detailed interaction without overwhelming the user. Additionally, the updated design separates data retrieval, event processing, and UI rendering responsibilities more clearly, improving maintainability, scalability, and overall usability.

This improvement will demonstrate my ability to meet several key course outcomes, particularly in designing and evaluating computing solutions. By identifying a limitation in the original system, I will have developed a redesigned interface that applies algorithmic principles such as sorting and structured data handling to ensure events are presented clearly and efficiently. In doing so, I will also have managed important design trade-offs between simplicity and functionality through the introduction of a layered approach that balances a clean monthly overview with a detailed daily view. This enhancement will also improve visual communication by ensuring that event data is no longer hidden and is instead presented in a clear, organized, and user-friendly manner. This reflects my ability to develop technically sound solutions that are appropriately adapted to user needs and context. Finally, the implementation of scalable UI patterns and improved interaction design demonstrates my ability to apply effective techniques

and tools to create software that delivers meaningful value and aligns with real-world application goals.

## **Category Two: Algorithms and Data Structures**

As part of prior Software Design and Engineering improvements to the CS-360: Mobile Architecture and Programming final project, a Daily View was introduced to support detailed event visualization. For an enhancement related to Algorithms and Data Structures, I will build upon this functionality through the implementation of an interval-based layout algorithm for organizing events within the Daily View of the event calendar application. Although the system will have already been able to retrieve and display multiple events for a given day, without these enhancements, it will not be able to properly handle overlapping time intervals. As a result, events occurring at the same time may visually overlap, reducing clarity making individual event interaction difficult.

To address this issue, I will implement an interval partitioning algorithm that organizes events based on their temporal relationships. Events will first be sorted by their start time, before being iteratively assigned to the earliest available column in which no temporal conflict exists. If no suitable column is available, a new column will be created. This will ensure that overlapping events are consistently separated spatially within the interface and remain independently accessible.

This enhancement replaces the previous linear rendering approach with a structured interval scheduling and visualization model. By processing events according to time-based

constraints rather than sequential insertion, the system is able to more effectively support concurrent events while maintaining scalability and visual clarity.

**Interval Partitioning Pseudocode:**

```
events = getEventsForDay(day)
sort events by start_time

columns = []

for each event in events:
    placed = false

    for each column in columns:
        lastEvent = last event in column

        if event.start_time >= lastEvent.end_time:
            add event to column
            placed = true
            break

    if placed == false:
        create new column
        add event to new column
        add column to columns

// Assign layout positions
totalColumns = number of columns

for each columnIndex in columns:
    for each event in column:
        event.width = 1 / totalColumns
        event.x_position = columnIndex / totalColumns
```

This enhancement primarily demonstrates my ability to design and evaluate computing solutions using algorithmic principles. The implementation of an interval-based layout algorithm applies established interval partitioning techniques to address the problem of overlapping time-based event data, while managing trade-offs between computational simplicity and visual clarity.

In addition, this work demonstrates the application of well-founded algorithmic methods and software design techniques to implement a functional and scalable solution within a mobile application. The integration of event scheduling logic with user interface rendering reflects the use of appropriate computing tools and practices to improve overall system functionality. Furthermore, this enhancement supports professional-quality visual communication by improving the clarity and coherence of event representation. By restructuring how event data is presented, the system provides a more intuitive and interpretable visualization of time-based information for the user.

### **Category Three: Databases**

For the database-focused enhancement, I will once again build upon the Event Calendar application developed for my CS-360: Mobile Architecture and Programming final project. Specifically, I will redesign the application's model layer to utilize a cloud-based database system (Firebase Firestore) in order to support authenticated, multi-user access while enforcing secure data separation. In contrast to the existing SQLite-based local storage approach, this cloud-based solution enables centralized data persistence and rule-based access control.

Although SQLite provides a lightweight and reliable solution for local data storage, particularly for single-user applications, it presents limitations when applied to a more scalable, multi-user system. Its primary advantage in this project's context is its simplicity, offering direct access to structured data while maintaining basic data privacy through locally managed user records. However, this local-first approach limits the application's ability to effectively support



multiple authenticated users within a shared application environment. Additionally, because SQLite operates as a file-based local storage system, application data is tied directly to the device. As a result, uninstalling or resetting the application can lead to complete data loss, reducing persistence and reliability across devices. Extending functionality to more advanced features, such as cloud-synced calendars or email-triggered event updates, also become significantly more complex without a centralized, network-accessible database architecture. For these reasons, a cloud-based database solution such as Firebase Firestore provides a more suitable foundation for the enhanced system requirements.

To implement this transition, the application's data flow would be restructured from a local file-based storage model to a cloud-hosted document-based structure. Event data would be stored within user-specific collections in Firebase Firestore, allowing each authenticated user to maintain an isolated set of calendar events. User authentication would serve as the entry point to the system, ensuring that all database interactions are associated with a verified user identity. Once authenticated, the application would retrieve and update event data through structured cloud queries rather than direct local database access. This shift would require refactoring the existing model layer to separate data storage concerns from application logic, ensuring that all event operations are routed through the cloud database interface.

This database-focused enhancement primarily demonstrates my ability to develop systems that incorporate a security-focused mindset through the use of authentication and rule-based access control. By enforcing user-specific data isolation within a cloud-based database system, the design anticipates and mitigates unauthorized access to sensitive event data, ensuring secure data handling across the application. Additionally, this enhancement enables a multi-user architecture in which multiple users can interact with the same application while maintaining

separate and consistent data contexts, demonstrating my ability to build a shared system environment that supports diverse user interactions. Finally, this enhancement demonstrates the use of well-founded, industry-relevant tools such as cloud-based database technologies to improve scalability and system flexibility.

#### **IV. ePortfolio Overall Skill Set**

Overall, this ePortfolio is intended to demonstrate my growth and development within the Computer Science program through the enhancement of a previously developed mobile application. Across the included artifacts, narratives, and code review, the portfolio will highlight my abilities in software design, algorithmic problem-solving, database architecture, and professional communication. Collectively, these enhancements will demonstrate both technical competency and the ability to evaluate and improve an existing software system using industry-relevant development practices.

The code review portion of the ePortfolio is intended to demonstrate my ability to analyze and explain technical implementations within a software project. Through walkthroughs of the application's architecture, database redesign, and algorithmic enhancements, the code review will illustrate my understanding of software development practices, computational problem-solving, and system organization. This review will also demonstrate my ability to communicate technical concepts in a structured and professional manner.

Meanwhile, the accompanying narratives are intended to demonstrate my ability to evaluate software systems and justify design decisions through written technical communication.

These narratives explain the reasoning behind each enhancement area including user interface redesign, interval-based event organization, and cloud-based database integration. Through these written reflections, the ePortfolio will demonstrate both my technical understanding and my ability to communicate complex development concepts clearly and professionally.

Lastly, the professional self-assessment section is intended to demonstrate my overall growth throughout the Computer Science program and my preparedness for professional software development work. By reflecting on the technical and design-oriented skills developed through coursework and project implementation, this assessment will highlight my ability to apply foundational computing concepts to practical software engineering problems. It will also demonstrate the progression of my interests toward software development and application design within the field of Computer Science.